CE4011 Assignment 3 Report

Imran Shahriar

April 12, 2026

Contents

1 Object-Oriented Structural Analysis Program **2**
1.1 Introduction . . . 3
1.2 Analysis and Design Decisions . . . 4
1.3 UML Class Diagram . . . 5
1.4 Analysis Assumptions . . . 10
1.5 Input Format . . . 11
1.6 Program Workflow . . . 13
1.7 Output Format . . . 14
1.8 GitHub README Summary . . . 15

2 Testing and Verification **17**
2.1 Testing and Verification Framework . . . 18
2.2 Unit Tests . . . 19
2.3 Interface Tests . . . 20
2.4 Regression Tests . . . 21
2.5 Summary of Verification Results . . . 22

3 Warning and Error Handling **23**
3.1 Robustness Strategy . . . 24
3.2 Structure Sample (a) . . . 25
3.3 Structure Sample (b) . . . 26
3.4 Structure Sample (c) . . . 27
3.5 Structure Sample (d) . . . 28
3.6 Structure Sample (e) . . . 29
3.7 Discussion . . . 30

Chapter 1

Object-Oriented Structural Analysis Program

1.1 Introduction

This project develops a 2D structural analysis program using the Direct Stiffness Method (DSM) with an object-oriented design. The implementation supports frame and truss elements in a unified 3-DOF-per-node formulation (u_x, u_y, r_z) and computes nodal displacements, support reactions, and member-end forces under static loading.

The main objective is to provide a reusable and extensible solver architecture for CE4011 assignments, while preserving numerical efficiency by reusing the matrix library from Q1.

1.2 Analysis and Design Decisions

Why Object-Oriented Programming (OOP)

OOP was selected to separate responsibilities and improve maintainability:

- **Modeling clarity:** classes such as `Node`, `Element`, and `Structure` map directly to engineering entities.
- **Extensibility:** new element types or load models can be added by subclassing without redesigning the full solver.
- **Encapsulation:** geometry, stiffness, load conversion, and post-processing logic stay close to the associated class.

Why Direct Stiffness Method (DSM)

DSM is the standard finite-element formulation for linear structural analysis:

- naturally supports mixed element assemblies,
- gives a systematic global matrix equation $\mathbf{Kd} = \mathbf{F}$,
- aligns with sparse matrix assembly and iterative solution techniques.

Why Reuse the Q1 Matrix Library

The Q1 implementation (symmetric sparse matrix, vector, and conjugate gradient solver) was reused because:

- stiffness matrices are symmetric by formulation,
- sparse storage reduces memory usage,
- conjugate gradient is appropriate for large symmetric systems,
- it preserves continuity between Q1 and Q2 deliverables.

Why a Frame/Truss Hierarchy

A base `Element` abstraction defines shared behavior (geometry transforms, global mapping), while subclasses implement element-specific mechanics:

- `FrameElement`: axial + flexural stiffness, optional end releases, member-load fixed-end effects.
- `TrussElement`: axial-only stiffness embedded in the same 6-DOF element format.

This avoids duplicate code and preserves a consistent assembly interface.

1.3 UML Class Diagram

The architecture of the structural analysis program is organised into three logical layers: a **numerical library layer** (`SymmetricSparseMatrix`, `Vector`, `ConjugateGradientSolver`), a **model layer** (`Node`, `Material`, `Section`, `Element`, `FrameElement`, `TrussElement`), and an **analysis coordination layer** (`Structure`). The full class diagram is presented in Figure 1.1, and each relationship is described in detail below.

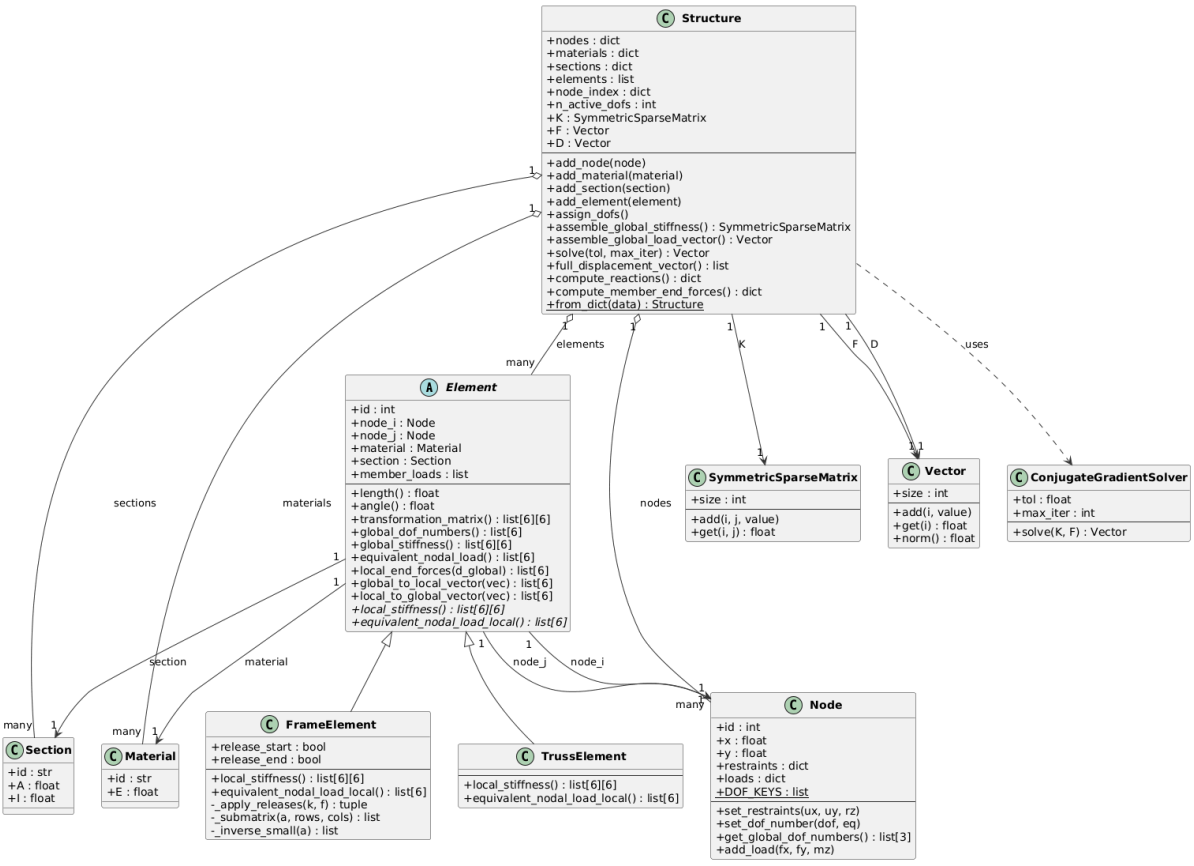

Figure 1.1: UML class diagram of the structural analysis program

Inheritance and Abstraction

`Element` is defined as an **abstract base class**. It declares two abstract methods that every concrete element type must implement: `local_stiffness()` and `equivalent_nodal_load_local()`. All remaining behaviour — coordinate transformation, global stiffness assembly, equivalent load transformation, and member-end force recovery — is implemented once in `Element` and inherited by every subclass. This is a direct application of the **Template Method** pattern: the base class defines the algorithm skeleton while subclasses supply the element-specific steps.

`FrameElement` and `TrussElement` each participate in a **generalisation / specialisation** (is-a) relationship with `Element`:

- `FrameElement` **is-a** `Element`. It specialises the base class by implementing full axial and flexural stiffness, optional moment releases at either end via static condensation,

and fixed-end force vectors for transverse point loads and uniformly distributed loads.

- `TrussElement` **is-a** `Element`. It specialises the base class by restricting stiffness to the axial direction only. Rotational and transverse degrees of freedom receive zero stiffness contributions, and only axial member loads are accepted.

In UML notation both relationships are represented by a solid line with a hollow triangular arrowhead pointing from the subclass to the abstract parent, indicating that the subclass inherits the interface and all non-abstract behaviour of `Element`.

Encapsulation

Encapsulation is enforced at two levels throughout the design.

At the **element level**, the internal condensation helpers of `FrameElement` — `_apply_releases()`, `_submatrix()`, `_inverse_small()`, `_mat_mul()`, `_mat_sub()`, `_mat_vec()`, and `_vec_sub()` — are prefixed with an underscore to signal that they are private implementation details. External callers interact only with the public interface defined on `Element`. This prevents the release-condensation logic from leaking into the assembly or postprocessing routines.

At the **structure level**, `Structure` holds its internal state (`K`, `F`, `D`, `node_index`, `n_active_dofs`) privately and exposes behaviour only through well-defined methods (`assemble_global_stiffness()`, `solve()`, `compute_reactions()`, `compute_member_end_forces()`). No external code is required to manipulate the stiffness matrix or displacement vector directly.

Modularity

The program is partitioned into three independently maintainable modules.

- **matrixlib**: provides `SymmetricSparseMatrix`, `Vector`, and `ConjugateGradientSolver`. This module has no dependency on structural mechanics and can be reused for any symmetric linear system.

- **model**: contains all structural classes. `Element` and its subclasses depend on `Node`, `Material`, and `Section` for geometric and physical data, but have no knowledge of `Structure` or the numerical library.

- **analysis coordination**: `Structure` imports from both `model` and `matrixlib` but is not imported by either. This one-way dependency ensures that changes to the numerical solver or element formulations do not cascade into each other.

Class-by-Class Relationship Analysis

Node. `Node` is a **data-bearing entity** that stores coordinates, restraint flags, applied nodal loads, and global degree-of-freedom equation numbers. It has no dependency on any other class in the system. `Node` is referenced by `Element` (two associations) and owned in aggregate by `Structure` (aggregation). `Node` exposes `DOF_KEYS` as a class-level constant, making the degree-of-freedom ordering (`ux`, `uy`, `rz`) part of the class contract rather than a convention scattered across callers.

Material. `Material` is a simple value object holding the elastic modulus `E`. It participates in a **1:1 association** with each `Element` instance: one element references exactly one material. Multiple elements may reference the same `Material` object, making the relationship from `Material` to `Element` effectively **1:Many** across the model, though `Material` itself carries no back-reference. `Material` is stored and managed by `Structure`.

Section. `Section` holds the cross-sectional properties `A` and `I`. Its relationship structure is identical to that of `Material`: a **1:1 association** to each `Element` at the instance level, and a potential **1:Many** relationship across the full model when a single section is shared by multiple elements. `Structure` owns the section registry.

Element (abstract). `Element` participates in the following relationships:

- **Association to Node (1:1, twice)**. Each `Element` holds exactly one reference to `node_i` and one to `node_j`. These references are set at construction time and do not change, making them navigable associations rather than aggregations: `Element` uses the node but does not own it. The multiplicity from `Element` to each `Node` is therefore **Many:1** across the model, since many elements can share the same node.
- **Association to Material (Many:1)**. Many elements may share one material definition.
- **Association to Section (Many:1)**. Many elements may share one section definition.
- **Generalisation to FrameElement and TrussElement**. Described above under Inheritance.

The `member_loads` attribute is a Python list owned by each `Element` instance. It represents a **whole/part** (composition) relationship between `Element` and the load dictionaries: the loads have no existence independent of the element that carries them. In UML this is a filled diamond on the `Element` side.

FrameElement. Beyond the inherited relationships, `FrameElement` introduces two boolean attributes (`release_start`, `release_end`) that control internal behaviour via `_apply_releases()`. This method performs static condensation of the local stiffness matrix and the fixed-end force vector simultaneously when one or both moment releases are active. The condensation helpers are entirely internal: no other class calls them directly, preserving the encapsulation boundary.

TrussElement. `TrussElement` adds no new attributes beyond what `Element` provides. Its specialisation is purely behavioural: `local_stiffness()` returns a matrix with nonzero entries only at positions `(0,0)`, `(0,3)`, `(3,0)`, and `(3,3)`, and `equivalent_nodal_load_local(` rejects any non-axial load direction with an explicit error.

Structure. `Structure` is the central coordinator and participates in the largest number of relationships:

- **Aggregation of Node (1:Many)**. `Structure` owns a dictionary of `Node` objects. The multiplicity is **1:Many**: one structure contains many nodes. This is an aggregation (hollow diamond) rather than a composition because, in principle, `Node` objects could exist and be inspected independently of the `Structure` that manages them.
- **Aggregation of Material (1:Many)**. `Structure` maintains a material registry. Multiplicity is **1:Many**.
- **Aggregation of Section (1:Many)**. `Structure` maintains a section registry. Multiplicity is **1:Many**.
- **Aggregation of Element (1:Many)**. `Structure` owns the element list. Multiplicity is **1:Many**. This is the primary whole/part relationship in the model layer: elements are created for and managed by a specific structure.
- **Association to SymmetricSparseMatrix (1:1)**. After assembly, `Structure` holds one instance of `SymmetricSparseMatrix` as its global stiffness matrix `K`. Before `assemble_global_stiffness()` is called, `K` is `None`, so the association is conditional on the analysis state.
- **Association to Vector (1:1, twice)**. `Structure` holds `F` (load vector) and `D` (displacement solution), each a `Vector` instance, after the respective assembly and solve steps.
- **Dependency on ConjugateGradientSolver**. `Structure.solve()` instantiates a `ConjugateGradientSolver` locally, uses it to solve the system, and discards it. This is a **dependency** (dashed arrow) rather than an association, because the solver is not stored as an attribute of `Structure`. The relationship is **transient**: it exists only for the duration of the `solve()` call.
- **Static factory method `from_dict()`**. This class method acts as a **factory**: it receives a raw dictionary, constructs all `Node`, `Material`, `Section`, and `Element` instances, and returns a fully initialised `Structure`. This centralises object initialisation and is the only path through which JSON input enters the object model. In UML it is shown as a static method on `Structure` with a return type of `Structure`.

Hierarchy Summary

The class hierarchy has two levels of depth:

1. **Abstract root**: `Element` defines the element interface and shared geometry behaviour.
2. **Concrete leaves**: `FrameElement` and `TrussElement` implement element-specific mechanics.

No further subclassing is anticipated within the current scope. The hierarchy is intentionally shallow to keep the design navigable and to avoid the fragile-base-class problems that arise from deep inheritance chains.

Semantic Discrepancy Notes

Two naming inconsistencies exist between the module-level legacy functions in `frame_element.py` and the object-oriented interface:

- `compute_rotation_matrix()`, `compute_global_stiffness()`, `matrix_multiply()`, `matrix_transpose()`, and `print_matrix()` are defined at module level but are never called by `FrameElement` or `Element`. The active implementation resides in `Element.transformation_matrix()` and `Element.global_stiffness()`. These legacy functions are dead code and represent a semantic discrepancy between what the module exposes and what the class hierarchy actually uses. They should be removed before final submission to prevent confusion.
- The parameter named `R` in the legacy `compute_rotation_matrix()` function is mathematically the transformation matrix `T` (global-to-local), not a rotation matrix in the conventional sense. The object-oriented code avoids this ambiguity by naming the method `transformation_matrix()` and documenting its direction explicitly.

Multiplicity Summary Table

Table 1.1: Summary of class relationship multiplicities

From	**To**	**Type**	**Multiplicity**
FrameElement	Element	Generalisation	Many:1
TrussElement	Element	Generalisation	Many:1
Element	Node (node_i)	Association	Many:1
Element	Node (node_j)	Association	Many:1
Element	Material	Association	Many:1
Element	Section	Association	Many:1
Element	member_loads	Composition	1:Many
Structure	Node	Aggregation	1:Many
Structure	Material	Aggregation	1:Many
Structure	Section	Aggregation	1:Many
Structure	Element	Aggregation	1:Many
Structure	SymmetricSparseMatrix	Association	1:1
Structure	Vector (F)	Association	1:1
Structure	Vector (D)	Association	1:1
Structure	ConjugateGradientSolver	Dependency	transient

1.4 Analysis Assumptions

The current solver is based on the following assumptions:

- Linear elastic material behavior.
- Small displacement and small rotation kinematics.
- Static loading (no inertia or damping effects).
- 2D planar structural behavior.
- Euler-Bernoulli frame behavior (no shear deformation in frame elements).
- Truss elements carry axial force only.
- Supports/restraints are modeled as prescribed zero DOFs.

1.5 Input Format

The solver accepts input in JSON format. The input defines the structural model through nodes, materials, sections, elements, and loads.

The required fields are summarized below:

- **nodes**: List of nodes with coordinates and boundary conditions
- **materials**: Material properties (e.g., Young's modulus E)
- **sections**: Cross-sectional properties (A, I)
- **elements**: Element definitions including type, connectivity, and optional releases and loads
- **nodal_loads**: External loads applied directly at nodes

Example Input

```
{
  "nodes": [
    {"id": 0, "x": 0.0, "y": 0.0, "restraints": {"ux": true, "uy": true, "rz": true}}
    {"id": 1, "x": 0.0, "y": 1.0},
    {"id": 2, "x": 1.0, "y": 1.0}
  ],

  "materials": [
    {"id": "steel", "E": 2.1e11}
  ],

  "sections": [
    {"id": "frame_sec", "A": 0.01, "I": 1.0e-5},
    {"id": "truss_sec", "A": 0.01, "I": 0.0}
  ],

  "elements": [
    {
      "id": 1,
      "type": "frame",
      "node_i": 0,
      "node_j": 1,
      "material": "steel",
      "section": "frame_sec",
      "member_loads": [
        {"type": "udl", "direction": "local_y", "w": -1000}
      ]
    },
    {
      "id": 2,
      "type": "truss",
```

```
      "node_i": 1,
      "node_j": 2,
      "material": "steel",
      "section": "truss_sec"
    }
  ],

  "nodal_loads": [
    {"node": 2, "fx": 0.0, "fy": -1000.0, "mz": 0.0}
  ]
}
```

Notes

- Frame elements include axial, shear, and bending behavior.
- Truss elements carry axial forces only.
- Boundary conditions are defined using boolean flags for each DOF.
- Member loads include uniformly distributed loads and point loads.

1.6 Program Workflow

The computational workflow is:

1. Assign global equation numbers to all unconstrained DOFs.
2. Assemble global stiffness matrix $\mathbf{K}$ from element contributions.
3. Assemble global load vector $\mathbf{F}$ from nodal loads and member-load equivalents.
4. Solve $\mathbf{Kd} = \mathbf{F}$ using conjugate gradient.
5. Reconstruct full nodal displacement vector (including restrained DOFs as zero).
6. Compute support reactions from $\mathbf{K}_{full}\mathbf{d}_{full} - \mathbf{F}_{full}$.
7. Compute local member-end forces for each element.

1.7 Output Format

The solver produces structured output in the terminal and optionally writes results to a text file. The output includes model information, solution status, nodal displacements, support reactions, and member-end forces.

Example Output

```
--- Starting Analysis: example_case ---

Nodes: 4 | Elements: 3
Active Equations: 5
Semi-bandwidth: 4 | Full bandwidth: 7

Processing Load Case: LC1...
System solved successfully.
Results written to ./results/example_case_results.txt

Nodal Displacements:
  Eq 1:  2.469135802e-06
  Eq 2: -2.469135802e-06
  Eq 3:  0.000000000e+00
  Eq 4: -4.938271605e-06
  Eq 5:  2.469135802e-06

Support Reactions:
  Node 0: Rx= 0.000000e+00, Ry= 5.000000e+02, Mz= 0.000000e+00
  Node 3: Rx= 0.000000e+00, Ry= 5.000000e+02, Mz= 0.000000e+00

Member-End Forces (local coordinates):
  Element 1 | i-end: Nx= 0.0, Vy=-500.0, Mz= 0.0
            | j-end: Nx= 0.0, Vy=-500.0, Mz= 0.0
```

Error/Warning Output

For unstable or invalid structures, the solver produces informative diagnostic messages:

```
ERROR: Global stiffness matrix is singular or nearly singular.
The structure may be unstable due to insufficient restraints
or a rigid-body mechanism.
```

Notes

- Results are printed in scientific notation for numerical clarity.
- Member-end forces are reported in local element coordinates.
- Warnings are issued for disconnected structures or suspicious configurations.
- The solver does not crash; all invalid cases are handled gracefully.

1.8 GitHub README Summary

This section summarizes the repository structure, usage, and input format of the developed structural analysis software.

Repository Link

`https://github.com/ImranMETU/CE4011_Assignment3`

How to Run

The program can be executed using:

```
python main.py
```

This runs the built-in example. To analyze a custom structure, provide a JSON input file:

```
python main.py path/to/input.json
```

Project Structure

The repository is organized into the following main modules:

- **model/**: Contains core object-oriented classes such as `Structure`, `Node`, `Element`, `FrameElement`, and `TrussElement`. This module also handles global assembly, DOF assignment, solving, and post-processing.
- **matrixlib/**: Implements the custom numerical backend developed in Assignment 2, including a symmetric sparse matrix representation and vector operations.
- **fem/**: Provides auxiliary finite element utilities, including element formulations, transformations, and load handling.
- **tests/**: Contains unit, interface, and regression tests used for verification.

Example Input Format

The solver accepts input in JSON format. The structure must include definitions for:

- Nodes (coordinates and boundary conditions)
- Materials (e.g., Young's modulus)
- Sections (area and moment of inertia)
- Elements (type, connectivity, releases, and loads)
- Nodal loads

The dictionary schema presented in Section 1.5 can be directly used as a JSON input file.

Summary

The repository provides a complete implementation of a 2D structural analysis solver using the direct stiffness method. It supports frame and truss elements, member releases, distributed and point loads, and produces nodal displacements, support reactions, and member-end forces.

Chapter 2

Testing and Verification

2.1 Testing and Verification Framework

To ensure the correctness and robustness of the developed structural analysis software, a structured testing and verification strategy was adopted. The objective is to validate both the numerical implementation of the Direct Stiffness Method and the correct interaction between object-oriented components.

The testing framework is divided into three categories:

- **Unit Tests**: Verify the correctness of individual components, particularly element-level formulations such as stiffness matrices, transformation matrices, and equivalent nodal load vectors.
- **Interface Tests**: Validate the interaction between components, including global stiffness matrix assembly, DOF mapping, and load vector construction.
- **Regression Tests**: Ensure that the complete system produces accurate results for known structural problems, including frame, truss, and mixed frame-truss configurations.

All tests are automated and executed using a test suite. Numerical comparisons are performed using clearly defined tolerances to account for floating-point precision.

Tolerance Criteria

The following tolerances are used throughout the verification process:

- **Matrix comparisons**: 10^{-10} (exact analytical agreement expected)
- **Displacement and force comparisons**: 10^{-6} to 10^{-3} (depending on problem scale)
- **Equilibrium checks**: residual forces close to machine precision

Verification Strategy

Verification is performed by comparing computed results with:

- Analytical solutions from structural mechanics (e.g., cantilever beam, two-bar truss)
- Known closed-form expressions for stiffness and load vectors
- Physical consistency checks such as equilibrium of forces and expected deformation patterns

This multi-level testing approach ensures that both the mathematical formulation and software implementation are reliable.

2.2 Unit Tests

Unit tests were developed to verify the correctness of individual frame element formulations independent of the global system. These tests focus on validating the mathematical implementation of the element-level stiffness matrices, transformation procedures, and equivalent nodal load formulations.

The following unit tests were implemented:

- **Frame Local Stiffness Matrix Test**
  This test verifies that the 6×6 local stiffness matrix of a 2D frame element matches the analytical formulation. The matrix symmetry and key coefficients (axial and bending terms) are checked against theoretical values.

- **Frame Transformation / Global Stiffness Test**
  This test validates the transformation of the local stiffness matrix into global coordinates. Both horizontal and vertical member orientations are tested to ensure that stiffness contributions are correctly mapped into the global system.

- **Frame Equivalent Nodal Load Test**
  This test verifies the correctness of equivalent nodal load vectors for member loads, including uniformly distributed loads (UDL) and point loads. The computed nodal forces are compared against analytical fixed-end force formulations, ensuring correct sign convention and magnitude.

All unit tests are automated and use predefined tolerances to validate numerical accuracy.

2.3 Interface Tests

Interface tests were implemented to verify the interaction between element-level formulations and the global structural system. These tests ensure that stiffness matrices, load vectors, and degree-of-freedom (DOF) mappings are correctly assembled.

The following interface tests were conducted:

- **Small Frame Global Assembly Test**
  A simple multi-element frame structure is used to verify correct global stiffness matrix assembly. The test checks matrix dimensions, symmetry, and selected entries to confirm proper DOF mapping and element contribution.

- **Mixed Frame–Truss Assembly Test**
  A hybrid structure consisting of both frame and truss elements is used to validate compatibility between different element types. The test ensures that truss elements do not introduce rotational stiffness and that the global matrix remains consistent and physically meaningful.

These tests confirm that the global assembly procedure functions correctly for both homogeneous and heterogeneous structural systems.

2.4 Regression Tests

Regression tests were performed to validate the overall correctness of the structural analysis solver for complete systems. These tests compare computed results against known analytical or reference solutions.

The following regression tests were implemented:

- **Frame Structure Test (Fixed-Fixed Beam with Central Point Load)**
  This test evaluates a fully restrained frame element subjected to a central point load. The expected reactions and member-end forces are compared with analytical solutions to verify correct load handling and force recovery.

- **Truss Structure Test (Two-Bar Truss)**
  A simple two-bar truss system is analyzed to verify axial force computation, displacement results, and support reactions. The absence of bending moments and shear forces in truss members is also confirmed.

- **Frame–Truss Structure Test (Mixed System)**
  A combined structure with both frame and truss elements is analyzed to validate full system behavior. The test verifies displacement results, reaction forces, and correct separation of axial and bending behavior between element types.

All regression tests use clearly defined tolerances and serve as benchmarks to ensure that future modifications do not introduce errors.

2.5 Summary of Verification Results

All implemented unit, interface, and regression tests were executed successfully using automated testing procedures. The results indicate that the structural analysis program produces accurate and consistent results within the specified numerical tolerances.

The test suite confirms:

- Correct implementation of frame element stiffness, transformation, and load formulations
- Proper assembly of the global stiffness matrix for both frame and truss systems
- Accurate computation of nodal displacements, support reactions, and member-end forces
- Consistent behavior across frame, truss, and mixed structural configurations

These results provide strong validation of the correctness and robustness of the developed structural analysis program.

Chapter 3

Warning and Error Handling

3.1 Robustness Strategy

This chapter presents the Q3 robustness and diagnostic behavior of the developed solver. The objective is to ensure that invalid, unstable, or poorly defined structural models are detected before misleading numerical results are reported.

The global static analysis is based on the system

$$\mathbf{Kd} = \mathbf{F},$$

where the global stiffness matrix $\mathbf{K}$ must be properly assembled, sufficiently constrained, and numerically stable in order to obtain a unique and physically meaningful solution.

For consistency in the Q3 demonstration cases, a common set of material and loading properties is adopted. Unless otherwise stated, frame members are modeled using reinforced concrete with elastic modulus

$$E = 25 \times 10^9 \text{ Pa},$$

and section properties

$$A = 0.09 \text{ m}^2, \qquad I = 6.75 \times 10^{-4} \text{ m}^4.$$

For truss members, the cross-sectional area is taken as

$$A = 0.01 \text{ m}^2.$$

A characteristic member length of

$$L = 1.0 \text{ m}$$

is used in the example structures. Applied loading is kept simple and consists of either a concentrated vertical point load of

$$P = 10 \text{ kN}$$

or a uniformly distributed load of

$$w = 5 \text{ kN/m},$$

depending on the structural configuration.

To improve reliability, the solver applies a layered validation procedure before and during analysis:

- **Input checks:** detection of missing node IDs, invalid element references, and duplicate identifiers.
- **Property checks:** detection of nonphysical or nonpositive material and section properties, such as $E \leq 0$, $A \leq 0$, or invalid I values for frame elements.
- **Geometry checks:** detection of zero-length elements and inconsistent element definitions.
- **Stability and constraint checks:** detection of unconstrained rigid-body motion, disconnected structural components, and zero-stiffness active degrees of freedom.
- **Solver checks:** detection of non-convergence or singular and near-singular stiffness behavior.

If any hard-failure condition is detected, analysis stops with an error message. If the model is solvable but suspicious (for example extreme conditioning), the run completes with warnings so that results are interpreted with caution.

3.2 Structure Sample (a)

Model description: Stable 2D frame benchmark with adequate supports and physically valid properties.

Result or warning: Analysis completes successfully. No stability warning is produced. Reactions satisfy global equilibrium within tolerance.

Screenshot:

```
######################################################################
Running: q3_case_a_unstable_portal.json
######################################################################
--- Starting Analysis: q3_case_a_unstable_portal ---
Nodes: 4 | Elements: 3
Active Equations: 10
Semi-bandwidth: 6 | Full bandwidth: 11
Determinacy summary:
* Component 1: frame, unstable by 1 degree
Processing Load Case: q3_case_a_unstable_portal...
ERROR: Global stiffness matrix is singular or nearly singular.
The structure may be unstable due to insufficient restraints, hinge configuration, or disconnected DOFs.

######################################################################
```

Figure 3.1: Structure Sample (a) diagnostic output placeholder

3.3 Structure Sample (b)

Model description: Model with insufficient support conditions leading to rigid-body motion.

Result or warning: Mechanism warning is triggered. The global stiffness matrix is singular (or effectively singular), and displacements are not accepted as valid structural response.

Screenshot:

```
#######################################################################
Running: q3_case_b_disconnected_member.json
#######################################################################
--- Starting Analysis: q3_case_b_disconnected_member ---
Nodes: 4 | Elements: 2
Active Equations: 9
Semi-bandwidth: 6 | Full bandwidth: 11
Determinacy summary:
* Component 1: frame, statically determinate
* Component 2: frame, unstable by 3 degrees
WARNING: The model contains disconnected components. Determinacy was assessed per connected component.
WARNING: 2 disconnected sub-structures detected: {0,1}, {2,3}.
Each sub-structure is independently supported and solvable.
Processing Load Case: q3_case_b_disconnected_member...
ERROR: Global stiffness matrix is singular or nearly singular.
The structure may be unstable due to insufficient restraints, hinge configuration, or disconnected DOFs.

#######################################################################
```

Figure 3.2: Structure Sample (b) diagnostic output placeholder

3.4 Structure Sample (c)

Model description: Model containing at least one invalid element definition (for example zero-length member or invalid connectivity).

Result or warning: Pre-analysis validation fails and reports the specific element ID and failure reason. Assembly is blocked until the model definition is corrected.

Screenshot:

```
######################################################################
Running: q3_case_c_disconnected_substructure.json
######################################################################
--- Starting Analysis: q3_case_c_disconnected_substructure ---
Nodes: 5 | Elements: 3
Active Equations: 8
Semi-bandwidth: 5 | Full bandwidth: 9
Determinacy summary:
* Component 1: frame, statically indeterminate by 1 degree
* Component 2: frame, statically determinate
WARNING: The model contains disconnected components. Determinacy was assessed per connected component.
WARNING: 2 disconnected sub-structures detected: {0,1,2}, {3,4}.
Each sub-structure is independently supported and solvable.
Processing Load Case: q3_case_c_disconnected_substructure...
Nodal Displacements (active equations):
  Eq   1: -4.728034436e-06
  Eq   2: -2.624128168e-07
  Eq   3:  4.517797267e-06
  Eq   4: -4.728034436e-06
  Eq   5: -3.099847310e-06
  Eq   6: -7.407407407e-06
  Eq   7: -4.444444444e-07
  Eq   8:  9.876543210e-06
Support Reactions (all nodes, global directions):
  Node  0: Rx= 1.000000e+03, Ry= 1.590429e+03, Mz=-4.095712e+02
  Node  1: Rx=-5.074980e-10, Ry= 6.718648e-08, Mz= 2.954877e-09
  Node  2: Rx= 6.075425e-10, Ry= 4.095712e+02, Mz= 2.883652e-09
  Node  3: Rx= 1.000000e+03, Ry= 2.000000e+03, Mz=-5.000000e+02
  Node  4: Rx= 8.185452e-12, Ry= 3.682203e-08, Mz= 4.277467e-12
Member-End Forces (local coordinates):
  Element  1 | i-end: Nx= 5.904288e+02, Vy=-1.000000e+03, Mz=-4.095712e+02 | j-end: Nx=-5.904288e+02,
Vy=-1.004992e-10, Mz=-9.042884e+01
  Element  2 | i-end: Nx=-6.075425e-10, Vy=-4.095712e+02, Mz= 9.042884e+01 | j-end: Nx= 6.075425e-10,
Vy=-5.904288e+02, Mz= 2.883652e-09
  Element  3 | i-end: Nx= 1.000000e+03, Vy=-1.000000e+03, Mz=-5.000000e+02 | j-end: Nx=-1.000000e+03,
Vy=-8.185452e-12, Mz= 4.277467e-12
System solved successfully.
Results written to ./results/q3_case_c_disconnected_substructure_results.txt

######################################################################
```

Figure 3.3: Structure Sample (c) diagnostic output placeholder

3.5 Structure Sample (d)

Model description: Model with invalid or non-physical property values (such as non-positive E or A).

Result or warning: Property validation error is issued before solution. The corresponding material/section record is identified for correction.

Screenshot:

```
#######################################################################
Running: q3_case_d_unstable_truss.json
#######################################################################
--- Starting Analysis: q3_case_d_unstable_truss ---
Nodes: 3 | Elements: 2
Active Equations: 2
Semi-bandwidth: 2 | Full bandwidth: 3
Determinacy summary:
* Component 1: truss, statically determinate
Processing Load Case: q3_case_d_unstable_truss...
Nodal Displacements (active equations):
  Eq   1:  0.000000000e+00
  Eq   2: -4.192627458e-06
Support Reactions (all nodes, global directions):
  Node  0: Rx=-1.250000e+02, Ry= 2.000000e+03, Mz= 0.000000e+00
  Node  1: Rx= 0.000000e+00, Ry= 0.000000e+00, Mz= 0.000000e+00
  Node  2: Rx= 1.250000e+02, Ry= 2.000000e+03, Mz= 0.000000e+00
Member-End Forces (local coordinates):
  Element  1 | i-end: Nx= 8.385255e+02, Vy= 5.590170e+02, Mz= 0.000000e+00 | j-end: Nx=-8.385255e+02, Vy=
5.590170e+02, Mz= 0.000000e+00
  Element  2 | i-end: Nx= 8.385255e+02, Vy= 5.590170e+02, Mz= 0.000000e+00 | j-end: Nx=-8.385255e+02, Vy=
5.590170e+02, Mz= 0.000000e+00
System solved successfully.
Results written to ./results/q3_case_d_unstable_truss_results.txt

#######################################################################
```

Figure 3.4: Structure Sample (d) diagnostic output placeholder

3.6 Structure Sample (e)

Model description: Mixed frame-truss structure that is solvable but numerically sensitive (high condition number tendency).

Result or warning: Analysis converges with cautionary diagnostics. Solver convergence is achieved, but a warning indicates potential sensitivity to modeling assumptions and input perturbation.

Screenshot:

```
##########################################################################
Running: q3_case_e_internal_hinge_beam.json
##########################################################################
--- Starting Analysis: q3_case_e_internal_hinge_beam ---
Nodes: 4 | Elements: 3
Active Equations: 5
Semi-bandwidth: 3 | Full bandwidth: 5
Determinacy summary:
* Component 1: frame, statically indeterminate by 1 degree
Processing Load Case: q3_case_e_internal_hinge_beam...
Nodal Displacements (active equations):
  Eq   1:  2.469135802e-06
  Eq   2: -2.469135802e-06
  Eq   3:  0.000000000e+00
  Eq   4: -4.938271605e-06
  Eq   5:  2.469135802e-06
Support Reactions (all nodes, global directions):
  Node  0: Rx= 0.000000e+00, Ry= 5.000000e+02, Mz=-7.105427e-14
  Node  1: Rx= 0.000000e+00, Ry= 9.947598e-14, Mz=-1.563194e-13
  Node  2: Rx= 0.000000e+00, Ry= 1.136868e-13, Mz= 0.000000e+00
  Node  3: Rx= 0.000000e+00, Ry= 5.000000e+02, Mz= 4.263256e-14
Member-End Forces (local coordinates):
  Element  1 | i-end: Nx= 0.000000e+00, Vy=-5.000000e+02, Mz=-7.105427e-14 | j-end: Nx= 0.000000e+00,
Vy=-5.000000e+02, Mz=-9.947598e-14
  Element  2 | i-end: Nx= 0.000000e+00, Vy=-5.000000e+02, Mz=-7.105427e-14 | j-end: Nx= 0.000000e+00,
Vy=-5.000000e+02, Mz= 0.000000e+00
  Element  3 | i-end: Nx= 0.000000e+00, Vy=-5.000000e+02, Mz= 0.000000e+00 | j-end: Nx= 0.000000e+00,
Vy=-5.000000e+02, Mz= 4.263256e-14
System solved successfully.
Results written to ./results/q3_case_e_internal_hinge_beam_results.txt
```

Figure 3.5: Structure Sample (e) diagnostic output placeholder

3.7 Discussion

Table 3.1 summarizes which sample models are analyzable and which are classified as mechanisms/errors.

Table 3.1: Q3 analyzability summary for sample structures

Sample	**Main Issue Type**	**Analyzable?**	**Program Response**
(a)	Stable baseline model	Yes	Normal completion; no warning
(b)	Insufficient restraints / mechanism	No	Singular-stiffness mechanism warning; reject response
(c)	Invalid element geometry/connectivity	No	Pre-analysis validation error; assembly blocked
(d)	Non-physical material/section properties	No	Property validation error; solve step blocked
(e)	Numerically sensitive but stable model	Yes (with caution)	Converged solution with conditioning warning

In summary, samples (a) and (e) are analyzable, while (b), (c), and (d) are intentionally non-analyzable test cases that verify warning/error pathways. This fulfills Q3's requirement to document how unstable or invalid models are detected and reported.